```
// Case A --------------------
Invoice f = new Invoice( workOrders );
f.setDate( Dates.today() );

EntityManager em = emf.createEntityManager();
EntityTransaction trx = em.getTransaction(); trx.begin();

em.persist( f );
f.setDate( Dates.yesterday() );

trx.commit();
em.close();

f.setDate( Dates.tomorrow() );
Console.println( f.toString() );

em = emf.createEntityManager();
trx = em.getTransaction(); trx.begin();

em.merge( f );
f.setDate( Dates.today() );

trx.commit();
em.close();

em = emf.createEntityManager();
trx = em.getTransaction(); trx.begin();

f = em.find(Invoice.class, f.getId() );
f.setDate( Dates.today() );

trx.commit();
em.close();

em = emf.createEntityManager();
trx = em.getTransaction(); trx.begin();

f = em.merge( f );
em.remove( f );

trx.commit();
em.close();

// Case B --------------------------
EntityManager em = emf.createEntityManager();
EntityTransaction trx = em.getTransaction(); trx.begin():

Invoice f = em.find(Invoice.class, 123);
f.setDate( Dates.today() );

trx.commit();
em.close();

f.setDate( Dates.yesterday() );
Console.println( f.toString() );

em = emf.createEntityManager();
trx = em.getTransaction(); trx.begin();
```

```
59
60  em.merge( f );
61
62  trx.commit();
63  em.close();
64
65  // Case C --------------------------
66  EntityManager em = emf.createEntityManager();
67  EntityTransaction trx = em.getTransaction(); trx.begin():
68
69  Invoice f = em.find(Invoice.class, 123)
70
71  trx.commit();
72  em.close();
73
74  f.setDate( Dates.today() );
75
76  em = emf.createEntityManager();
77  trx = em.getTransaction(); trx.begin():
78
79  Invoice f2 = em.find(Invoice.class, 123)
80  Invoice f3 = em.merge( f );
81
82  assert( f == f2 );
83  assert( f2 == f3 );
84
85  trx.commit();
86  em.close();
87
88  // Case D --------------------------
89  EntityManager em = emf.createEntityManager();
90  EntityTransaction trx = em.getTransaction(); trx.begin():
91
92  Invoice f = em.find(Invoice.class, 123);
93  for(WorkOrder a: f.getWorkOrders() ) {
94      a.getVehicle().getVehicleType().setPricePerHour( 50.0 );
95  }
96
97  trx.commit();
98  em.close();
99
100 // Case E --------------------------
101 EntityManager em = emf.createEntityManager();
102 EntityTransaction trx = em.getTransaction(); trx.begin():
103
104 Invoice f = em.find(Invoice.class, 123);
105
106 trx.commit();
107 em.close();
108
109 for(WorkOrder a: f.getWorkOrders() ) {
110     Console.println( a.getVehicle().getPlate() );
111 }
112
113 // Case F --------------------------
114 EntityManager em = emf.createEntityManager();
115 EntityTransaction trx = em.getTransaction(); trx.begin():
116
```

```
117  Invoice f = em.find(Invoice.class, 123);
118  Set<WorkOrder> workOrders = f.getWorkOrders();
119
120  Query q = em.createQuery("select f from Invoice f where f.id = 123");
121  Invoice f2 = q.getSingleResult();
122
123  assert( f == f2 );
124  assert( f2.getWorkOrders().equals( workOrders ) );
125
126  trx.commit();
127  em.close();
128
129  // Case G ------------------------
130  package uo.ri.presentation;
131  public class ShowInterventionsAction {
132      Long id = 123; // Console.askForLong("Work order id")
133
134      Service s = Factory.getService();
135      WorkOrder a = s.findById( id );
136      for(Intervention i : a.getInterventions() ) {
137          Printer.print( i );
138      }
139  }
140
141  package uo.ri.business;
142  public class Service {
143      ...
144      public WorkOrder findById(Long id) {
145          EntityManager em = emf.createEntityManager();
146          EntityTransaction trx = em.getTransaction();
147          trx.begin():
148
149          WorkOrder a = em.find(WorkOrder.class, id);
150
151          trx.commit();
152          em.close();
153
154          return a;
155      }
156      ...
157  }
```